Gymnázium, Praha 6, Arabská 14 předmět
Programování, vyučující Daniel Kahoun

Počítačová simulace šachové partie proti umělé inteligenci

ročníkový projekt

Viktor Meca, II.F březen 2026

Poděkování

Rád bych poděkoval panu učiteli Ing. Danielu Kahounovi za poskytnuté konzultace k projektu a taky za jeho cenné rady a připomínky k mému kódu. Jeho znalosti a ochota mi pomoci byly klíčové pro zdárné dokončení projektu. Také bych rád poděkoval mému otci PhDr. Viktorovi Mecovi, Ph.D za jeho trpělivost a cenné rady při vytváření programu. Nemůžu opomenout zmínit mé kamarády, kteří mi neváhali, kdykoli, kdy jsem potřeboval, pomoci.

Anotace

Cílem tohoto projektu je navázat na předchozí ročníkový projekt, *Šachový program pro dva hráče,* a rozšířit jej o možnost hry proti umělé inteligenci. Program tak kromě klasické hry dvou hráčů nabídne také možnost hrát proti AI, vzdát partii nebo nabídnout remízu pomocí nově přidaných tlačítek. Celý program je zpracován v programovacím jazyce Java a jeho grafické rozhraní je vytvořeno pomocí knihoven Swing a AWT.

The aim of this project is to build on the previous year's project Chess Program for Two Players, and expand it with the possibility of playing against artificial intelligence. In addition to the classic two-player game, the program will also offer the possibility of playing against AI, giving up the game or offering a draw using newly added buttons. The entire program is written in the Java programming language and its graphical interface is created using the Swing and AWT libraries.

Ziel dieses Projekts ist es, auf dem letztjährigen Projekt „Schachprogramm für zwei Spieler“ aufzubauen und es um die Möglichkeit zu erweitern, gegen eine künstliche Intelligenz zu spielen. Neben dem klassischen Zwei-Spieler-Modus bietet das Programm nun auch die Option, gegen die KI anzutreten, das Spiel aufzugeben oder ein Remis anzubieten. Hierfür werden neue Schaltflächen benötigt. Das gesamte Programm ist in Java geschrieben, und die grafische Benutzeroberfläche wurde mit den Bibliotheken Swing und AWT erstellt.

Prohlášení

Prohlašuji, že jsem jediným autorem tohoto projektu, všechny citace jsou řádně označené a všechna použitá literatura a další zdroje jsou v práci uvedené. Tímto dle zákona 121/2000 Sb. (tzv. Autorský zákon) ve znění pozdějších předpisů uděluji bezúplatně škole Gymnázium, Praha 6, Arabská 14 oprávnění k výkonu práva na rozmnožování díla (§ 13) a práva na sdělování díla veřejnosti (§ 18) na dobu časově neomezenou a bez omezení územního rozsahu.

V Praze dne 31.3. 2026

Podpis autora: _____

Obsah

Úvod.....1
1.Struktura programu.....2
2. Spuštění programu.....4
2.1. Třída Chesswindow.....4
2.2 Ukázka GUI.....5
2.3 GUI pro nastavení pozice.....7
3. Generování legálních tahů.....9
4. Minimax.....11
4.1 Definice.....11
4.2 Princip fungování algoritmu.....11
4.3 Pseudokód minimaxu.....12
4.4 Implementace algoritmu v mém programu.....13
4.5 Optimalizace MiniMaxu.....14
4.6 0-63 reprezentace šachovnice.....17
4.6.1 Jak funguje indexování 0-63.....17
5. Piece- Square Tables.....19
6. Problémy.....21
Závěr.....21
Zdroje.....23

Zadání projektu

Vytvořte grafickou aplikaci pro hráče šachu umožňující hrát proti počítači s vizuálními efekty. Program naváže na stávající šachový program, který jsem vytvořil jako ročníkový projekt v 1. ročníku v jazyce Java 8 a prostředí NetBeans IDE 8.2. Program bude obsahovat logiku pro hraní umělou inteligencí prostřednictvím implementovaním příslušného algoritmu, který bude schopen zvolit nejlepší tah na základě prohledání herního stromu s určitou hloubkou. K vybrání nejlepší tahu bestMove v situacích, kdy nelze prohledat celý herní strom až do konce hry, bude používat ohodnocovací funkce. Tyto ohodnocovací funkce např. vypočítají materiální převahu pomocí součtu hodnot figur (např. Pěšec= 1, Jezdec = 3 a Dáma = 9) jednoho hráče a odečtením hodnoty figur soupeře.

Program se pokus implementovat také otáčení šachovnice, v případě že uživatel hraje za černé, aby tak vizuálně přizpůsobil šachovnici podle pohledu hráče. Program taky bude umět vizuální efekty pro legální tahy a nelegální tahy. Pro legální tahy použije bílou barvu a pro nelegální tahy červenou barvu.

Úvod

Tato ročníková práce navazuje na předchozí ročníkovou práci *Šachový program pro dva hráče*. Aplikace je implementována v jazyce Java, který používá verzi JDK 21, v prostředí Apache NetBeans IDE 29. Uživatelské rozhraní je postaveno na knihovně **Swing** (využitím třídy JFrame), zatímco pro vykreslování herní grafiky a manipulaci s grafickými prvky jsou využity prostředky knihovny **AWT**.

Při implementaci metod jsem narazil na řadu složitých problémů, se kterými jsem se dosud nesetkal. Proto jsem vycházel z řešení, kteří vymysleli odborníci na programování, které mi pomohli s opravou algoritmu Minimaxu (viz kapitola 4), správným získáním šachovnice pro AI a také se správným implementováním metod (např. calculateLegalMoves) pro získání všech možných tahů pro AI.

1.Struktura programu

Na vylepšení programu o možnost hrát proti AI bylo deklarováno více tříd. Každá třída má na starost jinou část programu. Třída Chesswindow se zabývá hlavním oknem programu JFrame s názvem “Chess Game”, které slouží pro vizualizaci hry jak pro dva hráče, tak pro AI a spuštěním herního vlákna.

Poté přidá třídu Chesswindowpanel do JFrame a získá standardní šachovnici s počátečním rozestavením figur pomocí metody createStandardBoard() ve třídě Chessboard. Chessboard musí při svém volání obsahovat v parametru její vnitřní třídu Builder, jelikož tato třída byla předána jako parametr jejímu konstruktoru.

Také spouští algoritmus Minimax s hloubkou 1, kterému předá šachovnici s základním postavením figur pomocí metody createStandardBoard(), která se nachází ve třídě pro stav, hodnocení a vykreslení šachovnice Chessboard. Třída původně měla umět vytvořit i tlačítko OnePlayer, naplnit panel nabídek (Menu bar) pomocí metody populateMenuBar() a také vytvořit a nakonfigurovat nabídku "Soubor" (File menu) v panelu nabídek aplikace.

Třída Chesswindowpanel se zabývá velikostí hlavního okna, vytvořením seznamu figurek, logikou hry, vykreslením šachovnice a figurek pomocí metody paintComponent(). Také používá konstanty WHITE a BLACK pro určení barvy hráče či figury. Třída Minimax se zabývá implementováním tohoto algoritmu. Dále byla třeba deklarovat třídu MoveStatus pro ověření, zda byl tah legální a zda nezanechal krále v šachu při pokusu o provedení tahu AI na šachovnici. Jakožto ohodnocovací funkce pro MiniMax posloužila třída SimpleBoardEvaluator, která implementuje rozhraní BoardEvaluator, která má pouze jednu abstraktní, ale důležitou metodu evaluate, která se používá v Minimaxu pro metody min() a max(), kde max() je pro hráče a min() je pro AI. Bylo nutné také zavést enum (výčtový typ) Alliance, která obsahuje dvě pojmenované konstanty WHITE a BLACK. Tento enum slouží stejně jako výše zmíněné konstanty v Chesswindowpanel. Bylo by dobré také zmínit třídu MoveTransition, která funguje na principu neměnnosti (immutability) a reprezentuje přechod mezi dvěma stavy šachovnice po provedení (nebo pokusu o provedení) konkrétního tahu. Místo toho, aby se přímo měnil stav stávající šachovnice, engine vytváří instanci MoveTransition, která uchovává informace o tom, jak se hra změnila

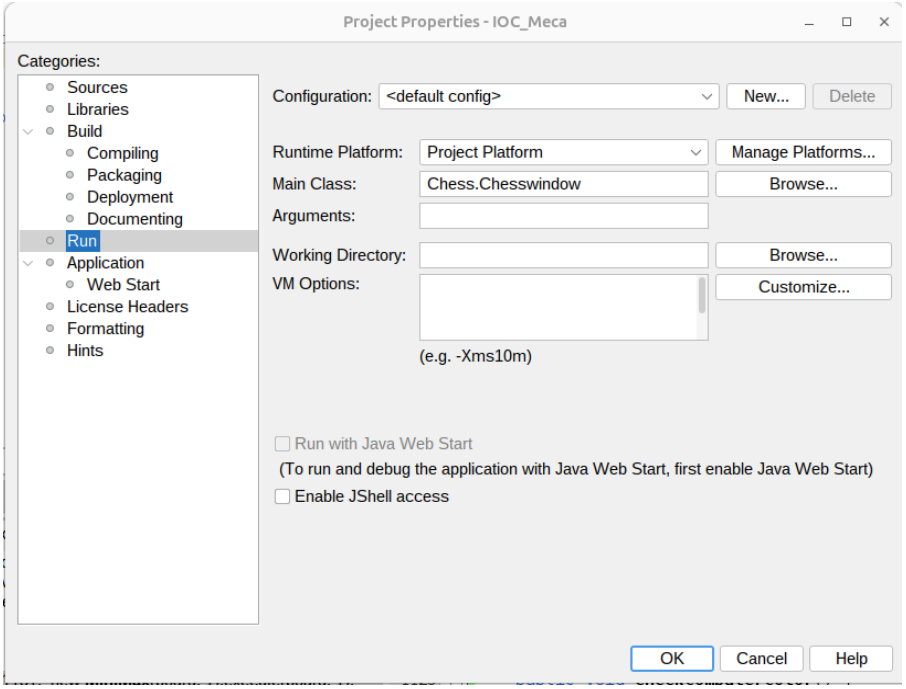
Tato třída Chessboard, která se zabývá stavem, hodnocením a vykreslením šachovnice, pomocí vnitřní třídy Builder nastaví pro AI základní postavení kamenů pomocí již výše zmíněné metody createStandardBoard().Spustí se také třída MiniMax prozatím s hloubkou 1 (pro testovací účely). Následně systém vypíše metodu toString() z Minimaxu, která obsahuje výpis hodnocení šachovnice boardEvaluator, hloubku prohledávání searchDepth, počet zhodnocených šachovnic boardsEvaluated a šachovnici board, která je zobrazena pomocí textové reprezentace, kde malá písmena jsou použita pro černé figury, např. p pro černého pěšce a velká písmena naopak pro bílé figury, např. K = bílý král.

Umělá inteligence bude hodnotit šachové pozice pomocí evaluační funkce a vyhledávat jeden z nejlepších tahů v omezené hloubce prohledávání pomocí algoritmu Minimax. Pro zvýšení efektivity by se dala využít optimalizace ve formě algoritmu Alfa-beta ořezávání, který zrychluje MiniMax tím, že eliminuje větve herního stromu, jež nemohou ovlivnit konečné rozhodnutí. Program by se dal určitě vylepšit implementovaným Menu, který by jednak vylepšil vzhled aplikace a taky umožnil uživateli vybrat, zda chce hrát s bílými nebo černými, atd.

2. Spuštění programu

2.1. Třída Chesswindow

Na spuštění programu byla použita již výše zmíněná třída Chesswindow, která musí být nastavená jako hlavní třída ve vlastnostech projektu (Project Properties) v kategorii Run.



Obrázek č.1: Ukázka správného nastavení Chesswindow jako hlavní třídu

Tato třída vytváří hlavní okno aplikace pomocí JFrame. Nastavením setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE) zajišťuje ukončení procesu při zavření okna a pomocí setResizable(false) fixuje jeho rozměry. Následně vytvoří instanci třídy Chesswindowpanel, který definuje preferovanou velikost okna a vykreslí po přidání panelu šachovnici a figury pomocí již výše zmíněné metody paintComponent().

Před samotným zobrazením se inicializuje standardní rozestavení figur metodou createStandardBoard() ze třídy Chessboard a spustí se algoritmus Minimax s hloubkou 1. Po přidání panelu do okna zajistí metoda pack() přesné přizpůsobení velikosti obsahu, setLocationRelativeTo(null) vycentruje okno na střed obrazovky a setVisible(true) jej zviditelní. Hra je následně zahájena voláním metody launchGame(). Níže je ukázka této metody:

```
public void launchGame() {

    gameThread = new Thread(this);

    gameThread.start();

}
```

2.2 Ukázka GUI

Jak je vidět z obrázku, tato metoda inicializuje a spouští nové vlákno gameThread. V něm běží metoda run(), která řídí herní smyčku. Pomocí proměnné nextDrawInterval vypočítává, kolik času uplynulo, pro překreslování snímků, čímž udržuje plynulou frekvenci **60 FPS**. Ukázka hlavní metody main v Chesswindow:

```
public static void main(String[] args) {
    JFrame frame = new JFrame("Chess Game");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setResizable(false);
    //Add GamePanel to the window
    Chesswindowpanel cp = new Chesswindowpanel();
    Chessboard board = Chessboard.createStandardBoard();
    MiniMax ai = new MiniMax(board,1);
    System.out.println(ai.toString());

    frame.add(cp);
    frame.pack();
    frame.setLocationRelativeTo(null);
    frame.setVisible(true);
    cp.launchGame();
}
```

..

Obrázek č. 2: Ukázka šachovnice s figurky po spuštění okna

Do GUI, jak je vidět z obrázku, byla přidána také již výše zmíněná tlačítka “Offer a draw”, “Set Position”, “Play vs PC” , “Give up” a “New Game”. Tyto tlačítka jsem vytvořil pomocí následujících metod createButton(), createButton2(), cretaeButton3(), createButton4() a createButton5() prostřednictvím třídy Swing. Tyto metody jsem pak zavolal pomocí metody initButtons() v konstruktoru třídy Chesswindowpanel. Níže je ukázka metody createButton4():

```
private void createButton4() {
    javax.swing.JButton compButton = new javax.swing.JButton("Play vs PC");
    compButton.setBounds(850, 650, 200, 50);
    compButton.setFont(new Font("Book Antiqua", Font.BOLD, 20));
    compButton.setFocusPainted(false);
    compButton.setBackground(new Color(200, 200, 200));

    compButton.addActionListener(e -> {
      isVsComputer = true; //režim proti PC
      promotion = false;
      gameover = false;
      currentColor = WHITE;
      restartGame(); //Resets the game and starts a new one with the vs PC settings

          if (currentColor == computerColor && !gameover && !stalemate &&
!draw) {
        new Thread(() -> {
          try {
            Thread.sleep(500);
          } catch (InterruptedException ex) {
            throw new RuntimeException(ex);
          }
          performAIMove();
          repaint();
        }) .start();
      }
            javax.swing.JOptionPane.showConfirmDialog(this, "You play against
computer");
    });
    this.add(compButton);
  }
```

Tato metoda inicializuje režim hry proti počítači nastavením logické proměnné isVsComputer na true a resetováním stavových proměnných promotion a gameover na false, čímž připraví herní prostředí pro nový zápas. Skrze volání restartGame() se spustí nová hra s výchozí barvou *WHITE*. Klíčovou součástí je spuštění samostatného vlákna pro AI, které po krátké prodlevě (zajišťující stabilitu uživatelského rozhraní) vykoná tah pomocí performAIMove() a následně překreslí rozhraní, čímž předchází zamrznutí aplikace. Pokud je v kódu definována hodnota computerColor = *WHITE*, počítač automaticky přebírá bílé figury a díky podmínce v této metodě zahájí hru prvním tahem ihned po stisknutí tlačítka, aniž by čekal na aktivitu uživatele.

2.3 GUI pro nastavení pozice

Pro případ, že by si uživatel chtěl nastavit pozici podle vlastní potřeby, bylo potřeba implementovat do hlavního okna programu již výše zmíněné tlačítko “Set position” pomocí metody createButton5(). Níže je ukázka této metody:

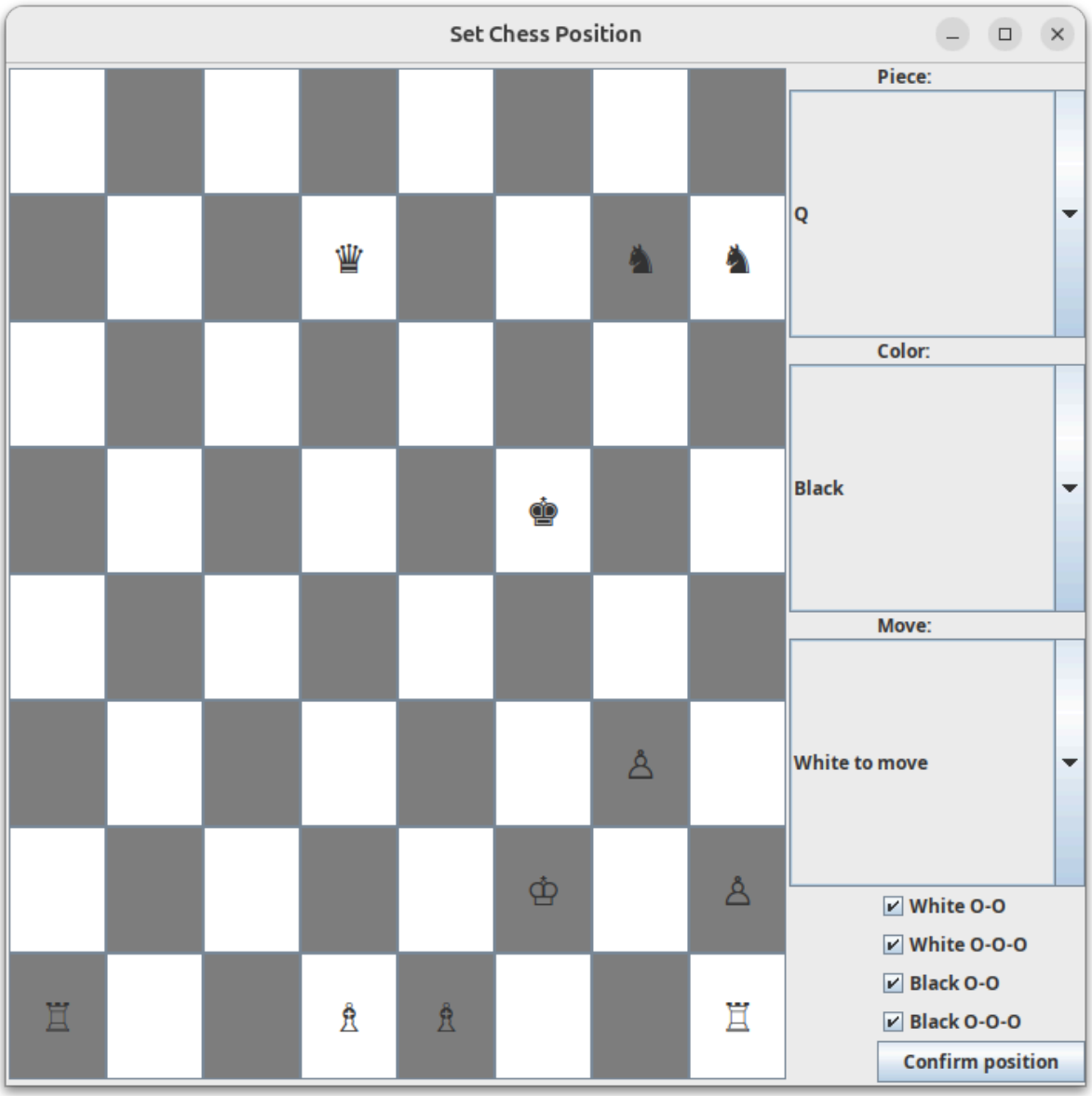
```
private void createButton5() {
    javax.swing.JButton setUpButton = new javax.swing.JButton("Set position");
    setUpButton.setBounds(850, 300, 200, 55);
    setUpButton.setFont(new Font("Book Antiqua", Font.BOLD, 18));
    setUpButton.setFocusPainted(false);
    setUpButton.setBackground(new Color(200, 200, 200));
    this.setLayout(null);
    this.add(setUpButton);

    setUpButton.addActionListener(e -> {
      ChessPositionSetter setter = new ChessPositionSetter(this);
      setter.setVisible(true);
    });
}
```

Tato metoda inicializuje a konfiguruje grafické tlačítko, které pojmenovala “Set position”, u kterého je pomocí metody setBounds() definováno pevné umístění a velikost. Aby bylo možné tyto souřadnice přesně určit, je v metodě volán příkaz this.setLayout(null), který vypíná automatický správce rozvržení a místo toho používá absolutní pozicování (tzv. Absolute Positioning), což označuje stav, kdy uživatel nepoužívá žádný správce rozvržení (Layout Manager) a místo toho ručně definuje přesné souřadnice a velikost každého prvku . Tento přístup uživateli umožní precizní kontrolu nad vizuálním uspořádáním prvků v ovládacím panelu bez vlivu výchozích algoritmů knihovny Swing.

Po kliknutí na tlačítko se v rámci obsluhy událostí vytvoří nová instance třídy ChessPositionSetter. Tato třída, dědící od JDialog, implementuje interaktivní editor šachových pozic, který uživateli umožňuje vizuálně sestavit rozvržení figur na šachovnici před zahájením hry. Vnitřní logika třídy využívá mřížku tlačítek reprezentujících herní pole, přičemž levým kliknutím myši se figura umístí a pravým odstraní. Součástí rozhraní je ovládací panel pro výběr typu figury, její barvy a určení hráče, který je v dané pozici na tahu. Třída rovněž spravuje stavy pro možnosti rošády na obou stranách a po potvrzení pozice přenáší všechna data zpět do hlavního panelu aplikace, kde se aktualizuje herní stav a seznam aktivních figur.

Klíčovým aspektem třídy ChessPositionSetter je komplexní validace šachových pravidel, která zajišťuje, že uživatelem sestavená pozice je legální a hratelná. Systém v reálném čase monitoruje počty figur a s ohledem na možnost proměny až osmi pěšců dovoluje umístit až 10 kusů od každé lehké či těžké figury (jezdec, střelec, věž) a maximálně 9 dam pro každou stranu, přičemž počet pěšců je limitován na 8 a jejich umístění na první či osmou řadu je striktně zakázáno. Validátor dále vyžaduje přítomnost právě jednoho krále od každé barvy a skrze metodu applyPosition() volá pomocnou logickou metodu isKingInCheckInPosition(), která ověřuje, zda se král strany, která není na tahu,

nenachází v nepřípustném šachu. Tento robustní kontrolní mechanismus efektivně brání vzniku nevalidních herních situací, čímž zajišťuje stabilitu šachového enginu a integritu následného výpočtu tahů pomocí MiniMaxu (viz kapitola 4).



Obrázek č.3: Ukázka GUI pro nastavení pozice uživatelem

3. Generování legálních tahů

K nalezení nejlepšího tahu prostřednictvím MiniMaxu bylo potřebovat implementovat funkce, které pro daný stav na šachovnici vygenerují všechny platné (legální) tahy, které může hráč v danou chvíli udělat. K tomuto účelu se v již výše zmíněné třídě pro stav, hodnocení a vykreslení šachovnice Chessboard museli vytvořit tyto tři metody: getAllLegalMoves(), calculateLegalMoves(), calculateActiveIndexes().

Ale pro Minimax jsem nakonec použil metodu getLegalMoves() v abstraktní třídě pro hráče Player, která vrací legalMoves, která představuje neměnnou kolekci všech platných tahů, které může hráč v daném stavu šachovnice provést. Tato kolekce je klíčovým prvkem pro řízení logiky hry, neboť v sobě kombinuje základní pohyby jednotlivých figur, speciální tahy typu rošády (přidáváné abstraktní metodou calculateKingCastles()) a zároveň slouží také jako filtr pro ověření stavu šachovnice. Výpočet obsahu této proměnné probíhá již při inicializaci objektu Player, což umožňuje efektivní zjišťování herních stavů, jako jsou mat (pomocí logické metody isInCheckmate()) nebo pat (pomocí logické metody isInStaleMate()), skrze pomocnou logickou metodu isTrapped(), která zjišťuje, zda má hráč nemá žádný legální tah. Pokud žádný neexistuje, hráč je zablokovaný (nemá žádný legální tah, např. v případě matu či patu). Díky zapouzdření (encapsulation) pomocí metody Collections.unmodifiableCollection je zaručena integrita dat, čímž se předchází nechtěným zásahům do seznamu povolených tahů během zpracování herního cyklu. Níže je ukázka již výše zmíněné metody getLegalMoves():

```
public Collection<Move> getLegalMoves() {
    return this.legalMoves;
}
```

Ukázka ostatních metod použitých pro generování legálních tahů:

```
public Collection<Move> getAllLegalMoves() {
    return Stream.concat(this.whiteplayer.getLegalMoves().stream(),
            this.blackPlayer.getLegalMoves().stream())
            .collect(Collectors.toList());
  }
```

Tato metoda slouží ke generování kompletního seznamu všech přípustných tahů v aktuálním stavu hry, a to prostřednictvím spojení dílčích seznamů legálních tahů bílého (whitePlayer) i černého (blackPlayer) hráče pomocí mocného nástroje pro funkcionální a deklarativní zpracování kolekcí dat **Java Stream API**. Výsledná kolekce, vzniklá zřetězením (concat) individuálních proudů tahů obou stran a jejich následnou transformací zpět do formátu seznamu (kolecki prvků) List, poskytuje ucelený přehled o všech dostupných herních variantách na šachovnici, což je klíčové pro detekci koncových stavů (mat, pat) nebo pro potřeby algoritmů umělé inteligence při prohledávání stavového prostoru.

```
private Collection<Move> calculateLegalMoves(final Piece[] boardConfig,
        final int[] pieceIndexes)  {
    final Collection<Move> legalsMove = new ArrayList<>();
    for (final int piece_index : pieceIndexes) {
        Piece p = boardConfig[piece_index];
        if (p != null) {

            Collection<Move> pieceMoves = p.calculateLegalMoves(this);
            legalsMove.addAll(pieceMoves);

             System.out.println("Piece: " + p.getClass().getSimpleName() + " row=" +
p.row + " col=" + p.col);
            System.out.println("Moves: " + p.calculateLegalMoves(this));
        }
    }
    return legalsMove;
  }
```

Tato metoda slouží k hromadnému výpočtu všech dostupných tahů pro vybranou skupinu figur na základě aktuální konfigurace šachovnice a pole (Array) indexů jejich pozic. V rámci cyklu metoda prochází zadané indexy, identifikuje na nich přítomné objekty figur a pro každou z nich volá její specifickou logiku výpočtu tahů, které následně agreguje do výsledné kolekce legalsMove. Součástí implementace je také pomocný kontrolní výpis do konzole, který zobrazuje typ figury, její souřadnice a seznam nalezených tahů pomocí abstraktní metody calculateLegalMoves() ve třídě Player, což usnadňuje ladění stavu, kdy by metoda mohla vracet prázdnou kolekci.

Ukázka metody getLegalMoves() a abstraktní metody calculateLegalMoves() ve třídě Piece použitých pro generování legálních tahů pro jednotlivé figurky:

```
public Collection<Move> getLegalMoves(Chessboard board) {
    System.out.println("Legální tahy jsou:" + calculateLegalMoves(board));

    return calculateLegalMoves(board);
}

 public abstract Collection<Move> calculateLegalMoves(final Chessboard board);
```

Abstraktní metoda calculateLegalMoves() definuje povinné rozhraní pro výpočet všech pravidly povolených tahů konkrétní figury na aktuální šachovnici; její specifickou logiku (např. pohyb koně vs. střelce) implementují až jednotlivé podtřídy (např. Bishop, King,...). Metoda getLegalMoves() pak slouží jako veřejný obal (wrapper), který tento výpočet spouští a zároveň zajišťuje výpis nalezených tahů do konzole pro účely ladění.

4. Minimax

4.1 Definice

Minimax představuje jeden z nejpoužívanějších algoritmů v oblasti herní umělé inteligence, zejména u strategických her pro dva hráče s nulovým součtem (tzv. zero-sum game), jako jsou šachy, dáma nebo piškvorky. Jeho podstata spočívá v systematickém prohledávání tzv. herního stromu, tedy v simulaci všech možných budoucích tahů a jejich dopadů. Název algoritmu přesně odráží jeho vnitřní logiku: zatímco se aktivní hráč snaží najít cestu k maximalizaci svého vlastního zisku, zároveň předpokládá, že jeho soupeř bude hrát optimálně a pokusí se mu tento zisk co nejvíce minimalizovat. Ve výsledku tak Minimax nevybírá tah pouze podle okamžité výhody, ale hledá strategii, která mu zajistí nejlepší možný výsledek i proti bezchybně hrajícímu protihráči.

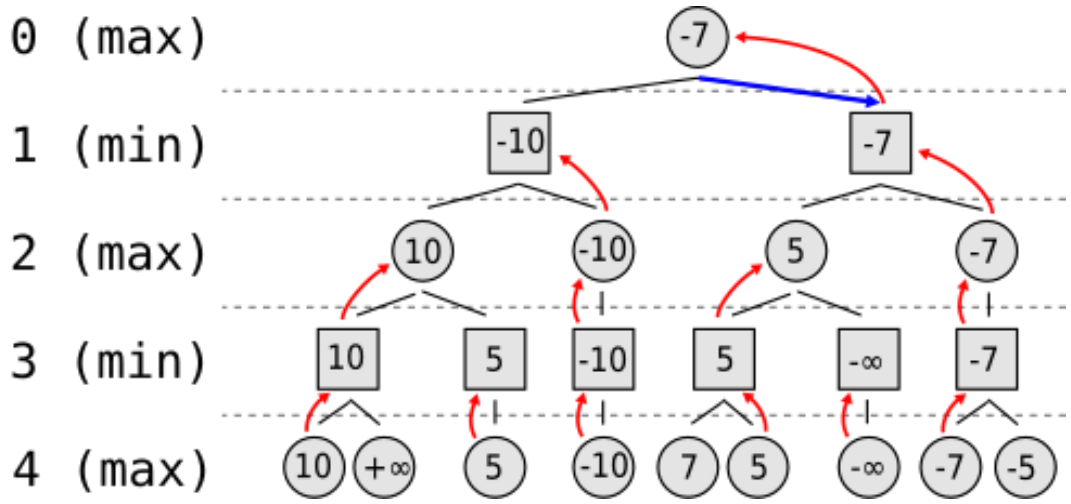
4.2 Princip fungování algoritmu

Hlavním úkolem algoritmu je objektivně vyhodnotit situaci na šachovnici a zvolit optimální pokračování. K tomu využívá tzv. ohodnocovací funkci, která převádí rozestavení figur na konkrétní číselnou hodnotu. V mém programu platí, že kladná čísla představují výhodu pro bílou stranu, zatímco záporná pro černou. Funkce při výpočtu zohledňuje především materiální rovnováhu (bodovou hodnotu zbývajících figur) a poziční faktory, jako je bezpečí krále nebo aktivita figur v centru. Algoritmus se pak v závislosti na tom, za jakou stranu právě „přemýšlí“, chová buď jako maximalizující hráč (snaží se skóre zvýšit), nebo jako minimalizující hráč (usiluje o co nejnižší, tedy zápornou hodnotu).

Protože však nejlepší tah v danou chvíli nemusí být výhodný z dlouhodobého hlediska, algoritmus nehodnotí pouze okamžitý stav, ale pomocí rekurze se snaží „dohlédnout“ o několik kroků dopředu. Vytváří si tak strom možných variant, kde simuluje tahy obou hráčů. Aby výpočet netrval příliš dlouho a počítač se nezacyklil, je hloubka tohoto prohledávání pevně omezena. Ve výsledku si pak program vybere ten tah, který mu i při nejlepší možné obraně soupeře zajistí co nejvyšší bodové hodnocení v konečné fázi výpočtu.

Na následujícím schématu (Obr. 4) je princip algoritmu demonstrován na modelu herního stromu s hloubkou čtyř úrovní. Celý proces začíná v nejnižší vrstvě (tzv. listové uzly), kde statická ohodnocovací funkce přiřadí konkrétním pozicím číselné hodnoty. V grafu si lze všimnout i hodnot $+\infty$ a $-\infty$, které v šachové logice představují tzv. terminální stavy – tedy situaci, kdy partie končí matem nebo jasnou výhrou/prohrou, kterou již nelze zvrátit. Tyto hodnoty se následně rekurzivně propagují směrem nahoru ke kořeni stromu (úroveň 0). V úrovních označených jako **max** si algoritmus vybírá větev s nejvyšším číslem, zatímco v úrovních **min** simuluje nejlepší tah soupeře výběrem hodnoty nejnižší. Modře zvýrazněná cesta v grafu ukazuje výsledné rozhodnutí: algoritmus nevolí tah s vidinou nejvyššího

možného zisku (např. 10), protože předpokládá, že by mu v tom racionálně hrající soupeř v úrovni 1 zabránil. Výsledkem je tedy volba nejlepšího „vynuceného“ výsledku, který je v dané hloubce propočtu dosažitelný (zde hodnota -7).



Obrázek č. 4: Princip fungování algoritmu

4.3 Pseudokód minimaxu

Ukázka pseudokódu pro implementaci algoritmu v Javě

```
funkce minimax(uzel, hloubka, jeMaximalizujiciHrac):
   // Základní případ: dosažení listu stromu nebo limitu prohledávání
   pokud hloubka == 0 nebo uzel je koncový:
      vrat ohodnocení(uzel)

   pokud jeMaximalizujiciHrac:
      nejlepsiSkore = -nekonečno
      pro každého potomka uzlu:
         skore = minimax(potomek, hloubka - 1, nepravda)
         nejlepsiSkore = max(nejlepsiSkore, skore)
      vrat nejlepsiSkore

   jinak (jeMinimalizujiciHrac):
      nejlepsiSkore = +nekonečno
      pro každého potomka uzlu:
         skore = minimax(potomek, hloubka - 1, pravda)
         nejlepsiSkore = min(nejlepsiSkore, skore)
      vrat nejlepsiSkore
```

Tato rekurzivní funkce implementuje rozhodovací algoritmus MiniMax pro hry dvou hráčů s nulovým součtem. Prohledává strom herních stavů do určené **hloubky**, přičemž střídavě maximalizuje zisk pro vlastního hráče a minimalizuje potenciální zisk pro protihráče (tzn. předpokládá, že protihráč zahraje svůj nejlepší možný tah) . Výsledkem je hodnota nejlepšího dosažitelného stavu při optimální hře obou stran.

4.4 Implementace algoritmu v mém programu

Ukázka implementace algoritmu v Javě pomocí metod min() a max() s pomocnou logickou metodou *isEndGameScenario*():

```
private int min(final Chessboard board, final int depth) {
   if (depth == 0 || isEndGameScenario(board)) {
      this.numBoardsEvaluated++;
      this.freqTable[this.freqTableIndex].increment();
      return this.boardEvaluator.evaluate(board, depth);
   }

   int lowestSeenValue = Integer.MAX_VALUE;
   for (final Move move : board.currentPlayer().getLegalMoves()) {
        final MoveTransition moveTransition =
board.currentPlayer().makeMove(move);
      if (moveTransition.getMoveStatus().isDone()) {
                                lowestSeenValue = Math.min(lowestSeenValue,
max(moveTransition.getToBoard(), depth - 1));
      }
   }
   return lowestSeenValue;
}

private int max(final Chessboard board, final int depth) {
   if (depth == 0 || isEndGameScenario(board)) {
      this.numBoardsEvaluated++;
      this.freqTable[this.freqTableIndex].increment();
      return this.boardEvaluator.evaluate(board, depth);
   }

   int highestSeenValue = Integer.MIN_VALUE;
   for (final Move move : board.currentPlayer().getLegalMoves()) {
        final MoveTransition moveTransition =
board.currentPlayer().makeMove(move);
      if (moveTransition.getMoveStatus().isDone()) {
                                highestSeenValue = Math.max(highestSeenValue,
min(moveTransition.getToBoard(), depth - 1));
      }
   }
   return highestSeenValue;
}

private static boolean isEndGameScenario(final Chessboard board) {
    returnboard.currentPlayer().isInCheckMate() ||
board.currentPlayer().isInStaleMate();
}
```

Rozhodovací jádro programu tvoří již výše zmíněné metody min() a max(), které prostřednictvím vzájemné rekurze simulují budoucí vývoj partie a umožňují tak umělé inteligenci „předvídat“ tahy soupeře do zvolené hloubky. Zatímco metoda max() usiluje o nalezení varianty s nejvyšším bodovým ziskem pro bílou stranu, metoda min() kalkuluje s nejlepší možnou odpovědí protihráče, který se logicky snaží tento zisk minimalizovat. Celý výpočetní proces je v každém kroku kontrolován pomocnou metodou isEndGameScenario(), která slouží jako efektivní pojistka – okamžitě ukončuje prohledávání dané větve, pokud v simulaci nastane mat nebo pat, čímž zajišťuje, že algoritmus pracuje pouze s reálnými herními situacemi a vrací přesné finální ohodnocení daného stavu na šachovnici. Pro efektivitu a přehlednost kódu se využívá standardní knihovní funkce Math.min() a Math.max(), které v každém kroku aktualizují dosud nejlepší nalezenou hodnotu. Součástí výpočtu je i inkrementace statistik pro sledování výkonu a počtu vyhodnocených pozic.

4.5 Optimalizace MiniMaxu

Hlavním cílem je zmenšit prohledávaný prostor herního stromu a rychleji vyhodnotit pozice. Příkladem takového vylepšení algoritmu je např. algoritmus ***Alfa-beta ořezávání.***

Alfa-beta ořezávání představuje nejznámější rozšíření minimax. Vrací stejný tah, ale “ožere” všechny větve (viz níže), kterou nemohou ovlivnit výsledný tah. Udává se, že oproti **minimaxu** dosáhne ve stejném čase až dvojnásobné hloubky prohledávání herních větví (propočtu).

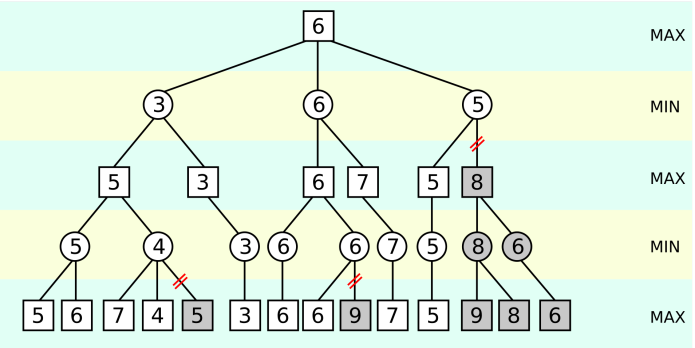
Aktuální verze programu využívá základní algoritmus minimax, který je s rostoucí hloubkou výpočtu limitován svou časovou náročností. V rámci studia problematiky jsem se proto věnoval možnostem optimalizace, zejména pomocí algoritmu **Alfa-beta ořezávání**. Tento algoritmus by umožnil ignorovat ty větve herního stromu, u kterých je již v průběhu výpočtu jasné, že nemohou vést k lepšímu výsledku než dříve prověřené varianty a tím pádem nemohou ovlivnit také konečné rozhodnutí algoritmu. Název je odvozen od mezních hodnot **alfa** a **beta**, které definují rozsah, v němž se hledá optimální skóre. Alfa je dolní mez (lower bound) a zabraňuje zbytečnému vygenerování tahů pro hráče, který má právě táhnout, a beta je horní mezí (higher bound) a zabraňuje vygenerování tahů protihráče. Alfa tak představuje nejlepší skóre, které si *maximalizující* hráč může zaručit, zatímco beta nejlepší skóre, které si *minimalizující hráč může zaručit.*

Dalším logickým krokem pro zvýšení výkonu by byla implementace **kaskádové metody** (iterative deepening) ve spojení s **transpozičními tabulkami**. Tento přístup by umožnil efektivnější řazení tahů a v ideálním případě by dovolil algoritmu dohlédnout ve stejném čase až dvakrát hlouběji. Ačkoliv tyto pokročilé techniky nejsou v současné verzi plně implementovány, jejich teoretická příprava tvoří základ pro budoucí rozšiřování schopností šachového enginu.

Pseudokód **Alfa-bety ořezávání:**

```
int alphabeta(int depth, Chessboard board, int alpha, int beta) {
   if (depth == 0 || isEndGameScenario(board))
      return evalFunction(board);
   for (Move m : possibleMoves(board)) {
      makeMove(board, m);
      int value = -alphabeta(depth - 1, board, -beta, -alpha);
      unmakeMove(board, m);
      if (value >= beta)
         return beta; // Beta-ořez (tah je příliš dobrý pro soupeře)
      if (value > alpha)
         alpha = value; // Aktualizace nejlepšího výsledku
   }
   return alpha;
}
```

Tato implementace využívá variantu **Negamax** (viz níže) s **Alpha-Beta ořezáváním**, která sjednocuje logiku obou hráčů do jediné funkce a díky parametrům α (minimum hráče) a β (maximum soupeře) eliminuje irelevantní větve stromu. Algoritmus využívá rekurzivní backtracking pro úsporu paměti a při optimálním řazení tahů dokáže prohledat až dvojnásobnou hloubku oproti standardnímu Minimaxu. Pokud hodnota tahu překročí hranici β, je větev oříznuta, protože ji racionální soupeř nikdy nepřipustí.



Obrázek č.5 Ukázka ořezání nepotřebných šedých větví v hloubce 4 uzlů

Dalším algoritmem, který by stálo za to v této podkapitole zmínit, je již výše zmíněný Negamax. Tento algoritmus vychází z Minimaxu, jen se liší ve formě implementace. Pro úlohu, kde se střídají strany na tahu a kde je možné vyjádřit stav jedním číslem, platí:

$$max(a,b) = -min(-a,-b)$$

Negamax se používá za účelem zjednodušení implementace minimaxu, protože jednou metodou je možné pokrýt jak tahy bílého, tak černého. Negamax tedy produkuje stejné výsledky jako Minimax. Pseudokód Negamaxu je následující:

```
int negamax(int depth, boolean isWhiteOnMove) {

   // Základní podmínka rekurze

   if (depth == 0 || isEndGameScenario(board)) {

      return evalFunction(isWhiteOnMove);

   }

   int maxValue = -INF;

   List<Move> moves = generateMoves();

   for (Move m : moves) {

      makeMove(m);     // Provede tah na herní desce

      // Rekurzivní volání s negací výsledku a změnou hráče

      int eval = -negamax(depth - 1, !isWhiteOnMove);

       undoMove(m);     // Vrátí stav desky zpět

       if (eval > maxValue) {

         maxValue = eval;

      }

   }

   return maxValue;

}
```

Tato implementace algoritmu **Negamax** sjednocuje ohodnocení pozic pro oba hráče do jediné rekurzivní funkce tím, že využívá principu nulového součtu, kde se hodnota aktuálního stavu rovná negaci nejlepšího dosažitelného výsledku soupeře v

následujícím tahu. Funkce prochází strom herních variant do stanovené hloubky a pomocí mechanismu **backtrackingu** (provedení a následné vrácení tahu) efektivně vyhodnocuje každou větev, přičemž v každém kroku maximalizuje dosažené skóre. Na rozdíl od základního Minimaxu je kód stručnější a elegantnější, tvoří ideální základ pro rozšíření o **Alpha-Beta ořezávání**, které by následně eliminovalo výpočetně zbytečné větve průchodu.

4.6 0-63 reprezentace šachovnice

Pro AI se nakonec použila reprezentace šachovnice pomocí čísel 0-63 (indexování polí) je základní metodou v počítačovém šachu, která transformuje dvourozměrnou desku (8x8) na jednorozměrnou řadu. Tato metoda se používá pro rychlou manipulaci s dat, zejména v technice zvané **bitboard**.

4.6.1 Jak funguje indexování 0-63

Šachovnice má 8 řad x 8 sloupců = 64 polí, takže počáteční index 0 obvykle odpovídá poli **a1** (levý dolní roh šachovnice z pohledu bílého) a konečný index 63 odpovídá poli **h8** (pravý horní roh z pohledu bílého), ale v mém případě je to přesně obráceně, takže pole **a1** odpovídá v mé reprezentaci číslu 56, takže poslední index 63 je v mém případě políčko **h1** (pravý dolní roh z pohledu bílého). Pole **h8** tak odpovídá číslu 7 v mé reprezentaci, takže počáteční index musí být o 6 polí doleva, takže se dostaneme na políčko **a8** (levý horní roh šachovnice z pohledu bílého).

Indexy zpravidla rostou zleva doprava, zdola nahoru (a1 = 0, b1 = 1, …, h1 = 7, a2 = 8,..., h8 = 63) ale v mém případě, to bude samozřejmě obráceně, což znamená, že to bude růst zleva doleva, shora dolů, (a8 = 0, b8 = 1,..., h8 = 7, h7= 8, …. h1 = 63).

V moderním šachovém programování se místo ukládání figur na konkrétní políčka (např. 8x8 pole) používá 12 samostatných 64bitových celých čísel (billboardů) – jeden pro každý typ figury a barvu (bílí pěšci, černí koně, atd.). Každý bit v 64bitovém čísle reprezentuje jedno pole šachovnice (0 = prázdné, 1 = figura přítomna). Tato reprezentace umožňuje velmi rychlé generování tahů pomocí bitových operací (AND,OR,XOR).

Základní rozestavení (0-63)

Při tomto indexování (a1=0, h8=63) jsou figury na začátku partie rozmístěny takto:

- **0-7 (1. řada):** Věž, Jezdec, Střelec, Dáma, Král, Střelec, Jezdec, Věž.
- **8-15 (2. řada):** 8 bílých pěšců.
- **48-55 (7. řada):** 8 černých pěšců.
- **56-63 (8. řada):** Věž, Jezdec, Střelec, Dáma, Král, Střelec, Jezdec, Věž (černé).

Ukázka metody *createStandardBoardImpl*(), která využívá 0-63 reprezentaci šachovnice:

```
public static Chessboard ***createStandardBoardImpl***() {

final *Builder* builder = new Builder();

 // black
builder.setPiece(new Rook(Alliance.*BLACK*, 0, false));
builder.setPiece(new Knight(Alliance.*BLACK*, 1, false));
builder.setPiece(new Bishop(Alliance.*BLACK*, 2, false));
builder.setPiece(new Queen(Alliance.*BLACK*, 3, false));
builder.setPiece(new King(Alliance.*BLACK*, 4, false));
builder.setPiece(new Bishop(Alliance.*BLACK*, 5, false));
builder.setPiece(new Knight(Alliance.*BLACK*, 6, false));
builder.setPiece(new Rook(Alliance.*BLACK*, 7, false));

for (int i = 8; i < 16; i++) {
builder.setPiece(new Pawn(Alliance.*BLACK*, i, false));
}

// white
for (int i = 48; i < 56; i++) {
builder.setPiece(new Pawn(Alliance.*WHITE*, i, false));
}

builder.setPiece(new Rook(Alliance.*WHITE*, 56, false));
builder.setPiece(new Knight(Alliance.*WHITE*, 57, false));
builder.setPiece(new Bishop(Alliance.*WHITE*, 58, false));
builder.setPiece(new Queen(Alliance.*WHITE*, 59, false));
builder.setPiece(new King(Alliance.*WHITE*, 60, false));
builder.setPiece(new Bishop(Alliance.*WHITE*, 61, false));
builder.setPiece(new Knight(Alliance.*WHITE*, 62, false));
builder.setPiece(new Rook(Alliance.*WHITE*, 63, false));

//white moves first
builder.setMoveMaker(Alliance.*WHITE*);

return builder.build();
}
```

5. Piece- Square Tables

Jednoduchý způsob přiřazení hodnot k určitým figurám na určitých políčkách. Tabulka je vytvořena pro každou figuru každé barvy, a hodnoty jsou přiřazeny každému políčku. Tento systém je rychlý, protože vyhodnocovací člen z piece-squares table lze postupně aktualizovat s tím, jak se v prohledávacím stromu provádějí a ruší tahy. Kvůli rychlosti, piece - square tables jsou výborným pomocníkem.

Níže je ukázka všech piece-square tables pro všechny figurky:

```
[ -3.0, -4.0, -4.0, -5.0, -5.0, -4.0, -4.0, -3.0],
[ -3.0, -4.0, -4.0, -5.0, -5.0, -4.0, -4.0, -3.0],
[ -3.0, -4.0, -4.0, -5.0, -5.0, -4.0, -4.0, -3.0],
[ -3.0, -4.0, -4.0, -5.0, -5.0, -4.0, -4.0, -3.0],
[ -2.0, -3.0, -3.0, -4.0, -4.0, -3.0, -3.0, -2.0],
[ -1.0, -2.0, -2.0, -2.0, -2.0, -2.0, -2.0, -1.0],
[  2.0,  2.0,  0.0,  0.0,  0.0,  0.0,  2.0,  2.0 ],
[  2.0,  3.0,  1.0,  0.0,  0.0,  1.0,  3.0,  2.0 ]
```

```
[ -2.0, -1.0, -1.0, -0.5, -0.5, -1.0, -1.0, -2.0],
[ -1.0,  0.0,  0.0,  0.0,  0.0,  0.0,  0.0, -1.0],
[ -1.0,  0.0,  0.5,  0.5,  0.5,  0.5,  0.0, -1.0],
[ -0.5,  0.0,  0.5,  0.5,  0.5,  0.5,  0.0, -0.5],
[  0.0,  0.0,  0.5,  0.5,  0.5,  0.5,  0.0, -0.5],
[ -1.0,  0.5,  0.5,  0.5,  0.5,  0.5,  0.0, -1.0],
[ -1.0,  0.0,  0.5,  0.0,  0.0,  0.0,  0.0, -1.0],
[ -2.0, -1.0, -1.0, -0.5, -0.5, -1.0, -1.0, -2.0]
```

```
[  0.0,  0.0,  0.0,  0.0,  0.0,  0.0,  0.0,  0.0],
[  0.5,  1.0,  1.0,  1.0,  1.0,  1.0,  1.0,  0.5],
[ -0.5,  0.0,  0.0,  0.0,  0.0,  0.0,  0.0, -0.5],
[ -0.5,  0.0,  0.0,  0.0,  0.0,  0.0,  0.0, -0.5],
[ -0.5,  0.0,  0.0,  0.0,  0.0,  0.0,  0.0, -0.5],
[ -0.5,  0.0,  0.0,  0.0,  0.0,  0.0,  0.0, -0.5],
[ -0.5,  0.0,  0.0,  0.0,  0.0,  0.0,  0.0, -0.5],
[  0.0,   0.0, 0.0,  0.5,  0.5,  0.0,  0.0,  0.0]
```

```
[ -2.0, -1.0, -1.0, -1.0, -1.0, -1.0, -1.0, -2.0],
[ -1.0,  0.0,  0.0,  0.0,  0.0,  0.0,  0.0, -1.0],
[ -1.0,  0.0,  0.5,  1.0,  1.0,  0.5,  0.0, -1.0],
[ -1.0,  0.5,  0.5,  1.0,  1.0,  0.5,  0.5, -1.0],
[ -1.0,  0.0,  1.0,  1.0,  1.0,  1.0,  0.0, -1.0],
[ -1.0,  1.0,  1.0,  1.0,  1.0,  1.0,  1.0, -1.0],
[ -1.0,  0.5,  0.0,  0.0,  0.0,  0.0,  0.5, -1.0],
[ -2.0, -1.0, -1.0, -1.0, -1.0, -1.0, -1.0, -2.0]
```

```
[-5.0, -4.0, -3.0, -3.0, -3.0, -3.0, -4.0, -5.0],
[-4.0, -2.0,  0.0,  0.0,  0.0,  0.0, -2.0, -4.0],
[-3.0,  0.0,  1.0,  1.5,  1.5,  1.0,  0.0, -3.0],
[-3.0,  0.5,  1.5,  2.0,  2.0,  1.5,  0.5, -3.0],
[-3.0,  0.0,  1.5,  2.0,  2.0,  1.5,  0.0, -3.0],
[-3.0,  0.5,  1.0,  1.5,  1.5,  1.0,  0.5, -3.0],
[-4.0, -2.0,  0.0,  0.5,  0.5,  0.0, -2.0, -4.0],
[-5.0, -4.0, -3.0, -3.0, -3.0, -3.0, -4.0, -5.0]
```

```
[0.0,  0.0,  0.0,  0.0,  0.0,  0.0,  0.0,  0.0],
[5.0,  5.0,  5.0,  5.0,  5.0,  5.0,  5.0,  5.0],
[1.0,  1.0,  2.0,  3.0,  3.0,  2.0,  1.0,  1.0],
[0.5,  0.5,  1.0,  2.5,  2.5,  1.0,  0.5,  0.5],
[0.0,  0.0,  0.0,  2.0,  2.0,  0.0,  0.0,  0.0],
[0.5, -0.5, -1.0,  0.0,  0.0, -1.0, -0.5,  0.5],
[0.5,  1.0, 1.0,  -2.0, -2.0,  1.0,  1.0,  0.5],
[0.0,  0.0,  0.0,  0.0,  0.0,  0.0,  0.0,  0.0]
```

Ukázka kódu v programu:

```
// Example Pawn Table: High values in the center and surroundings, low in the center on 2nd rank

public static final int [][] PAWN_TABLE = {
    {0,  0,  0,  0,  0,  0,  0,  0},
    {5,  5,  5,  5,  5,  5,  5,  5},
    {10, 10, 20, 30, 30, 20, 10, 10},
    {5,  5, 10, 25, 25, 10,  5,  5},
    {0,  0,  0, 20, 20,  0,  0,  0},
    {5, -5,-10,  0,  0,-10, -5,  5},
    {5, 10, 10,-20,-20, 10, 10,  5},
    {0,  0,  0,  0,  0,  0,  0,  0}
};
```

Tento kód představuje **Piece-Square Table (PST)** pro pěšce, což je základní heuristika používaná v šachových enginech k ohodnocení pozice figur na šachovnici. Dvourozměrné pole *PAWN_TABLE* přiřazuje každému poli specifickou bodovou hodnotu (bonus či penalizaci), která odráží strategickou výhodu daného umístění: například vyšší hodnoty ve středních řadách (20, 30) motivují engine k postupu pěšců vpřed a ovládnutí centra, zatímco záporné hodnoty na startovních polích (např. -20 u středových pěšců) simulují snahu o otevření prostoru pro vývin ostatních figur.

Při celkovém hodnocení pozice se pak tyto statické hodnoty přičítají k základní materiální ceně pěšce, což umožňuje programu činit pozičně informovaná rozhodnutí i bez hlubokého prohledávání stavového prostoru.

6. Problémy

Při vytváření nového programu s AI se vyskytlo hned několik problémů. Níže jsou uvedeny ty problémy, které jsou považovány za nejdůležitější:

- Indexová reprezentace šachovnice (pomocí sloupců a řad x 0 - 63 reprezentace)
- Špatný odkaz na metodu getLegalMoves() v Minimaxu (prázdné pole)
- Špatná inicializace Chessboard pro AI
- Špatný odkaz na šachovnici v Minimaxu (AI brala úplnou novou instanci šachovnici)
- Pohyb věže (AI si myslela, že může přeskakovat figurky)
- Král si vyhazuje vlastní pěšce (AI asi nevidí, že na políčko, kam se chystá jít černý král, je již pěšec)
- Proměnná pro výpočet pozice figurky piecePosition byla omylem nastavena na null
- Proměnná pro hráče, který je na tahu, ve třídě Player byla také omylem nastavena na null
- Neukončení partie (když AI dá mat, sice uživatel nemůže nic dělat, ale metoda paintComponent() ve třídě Chesswindowpanel nevykreslí na střed okna programu “White wins” nebo “Black wins” podle toho, kdo vyhrál a neukončí tak hru)

Co se týče prvního bodu, AI vůbec v podstatě v podstatě nedostala žádné figury s novou instancí 0-63 reprezentovanou šachovnici, a tak, když byla požádána o provedení tahu, tak vrátila NULL move (nic nezahrál), protože zkrátka nemohla nic zahrát kvůli výše uvedenému problému.

Dále jsem zjistil, že Minimax asi dostával špatný odkaz na metodu getLegalMoves(), protože mi pořád vracel již výše zmíněný NULL move, což bylo způsobeno tím, že metoda getLegalMoves() vracela prázdné pole, takže MiniMax logicky nemohl nic zahrát kvůli výše zmíněnému problému.

Při vytváření šachovnice jsem si neuvědomil, že musím zavolat již výše zmíněnou metodu createStandardBoardImpl(), která nejprve vytváří instanci vnitřní třídy Builder, poté přidá na příslušné souřadnice nejprve černé a pak bílé figury, do parametru metody setMoveMaker() vloží Alliance.*WHITE***,** aby AI věděl, že jako první bude táhnout bílý. Nakonec pomocí build(), což je **terminální metoda**, která vytvoří a vrátí cílový, často neměnný (immutable) objekt (v tomto případě figury na počátečních souřadnicích).

Závěr

Během vytváření programu jsem se inspiroval zkušeným programátorem Blackwidowem, který natočil sérii 50 videí o tom, jak naprogramovat šachový engine v Javě, která mi velice pomohla. Také jsem čerpal z jeho repozitáře na Githubu, kde měl uložené o něco novější zdrojové kódy, kterými jsem inspiroval při implementování např. MiniMax, šachovnice pro AI, heuristicke evaluační funkce pro MiniMax atd.

Program by se dal určitě vylepšit implementováním lepších herustických evaluačních funkcí, sjednocením šachovnice jak pro GUI (pro dva hráče), tak pro AI (**Minimax**).

Jak jsem již říkal, tak by **Minimax** mohl být vylepšen pomocí algoritmu ***Alfa-beta ořezávání***, který funguje jako optimalizační technika pro snižování počtu uzlů, které musí prohledávací algoritmus vyhodnotit v herním stromu. Používá se také v hrách dvou hráčů s nulovým součtem (tzv. zero-sum game) např. šachy, piškvorky a dáma. Zatímco čistý **minimax** prohledává všechny možné tahy, ***alfa-beta ořezávání*** eliminuje větve, u kterých je dokázáno, že neovlivní konečné rozhodnutí (tj. nemohou vést k lepšímu výsledku než již nalezená varianta).

Kvůli nedostatku času jsem bohužel nestihl implementovat různé obtížnosti botů, otáčení šachovnice pro černého hráče a přidání časomíry.

Program se mi celkem povedl, protože splňuje zadání projektu, tedy přes zmíněné chyby se mi podařilo implementovat Minimax pro AI a lze s ním hrát šachový zápas.

Zdroje

https://www.freecodecamp.org/news/simple-chess-ai-step-by-step-1d55a9266977/
https://stackoverflow.com/questions/59708531/chess-ai-implementation-in-java-project
https://github.com/Virksaabnavjot/Chess-AI
http://www.chesstantra.com/chess-facts/
https://is.slu.cz/th/zi4wd/FPF_DP_21_Algoritmus_Minimax_a_alfa-beta_orezavani_Juricek_Martin.pdf?lang=cs
https://is.muni.cz/th/aiw7t/bapr_digital.pdf?kodomez=termin-540379;lang=cs
https://www.youtube.com/playlist?list=PLOJzCFLZdG4zk5d-1_ah2B4kqZSeIlWtt
https://github.com/lhartikk/simple-chess-ai
https://www.geeksforgeeks.org/dsa/backtracking-algorithms/
https://portal.matematickabiologie.cz/index.php?pg=analyza-a-hodnoceni-biologickych-dat--vicerozmerne-metody-pro-analyzu-dat--volba-a-vyber-popisnych-promennych--selekce-promennych--algoritmy-selekce-promennych--algoritmus-min-max
https://ksvi.mff.cuni.cz/~dvorak/vyuka/NUIN017/Minimax.pdf
https://nlp.fi.muni.cz/uui/slajdy-2014/uui07-2.pdf
https://naos-be.zcu.cz/server/api/core/bitstreams/19ae03fc-5a48-4421-9249-9f0bf08a55fc/content
https://www.chessprogramming.org/Simplified_Evaluation_Function
https://github.com/amir650/BlackWidow-Chess
https://www.youtube.com/playlist?list=PLOJzCFLZdG4zk5d-1_ah2B4kqZSeIlWtt
https://www.inf.upol.cz/downloads/studium/PS/minimax.pdf
https://portal.matematickabiologie.cz/index.php?pg=analyza-a-hodnoceni-biologickych-dat--vicerozmerne-metody-pro-analyzu-dat--volba-a-vyber-popisnych-promennych--selekce-promennych--algoritmy-selekce-promennych--algoritmus-min-max
https://kam.fit.cvut.cz/bi-zum/media/lectures/09-minimax-v5.0-noanim.pdf
https://ksvi.mff.cuni.cz/~dvorak/vyuka/NUIN017/Minimax.pdf
https://www.youtube.com/watch?v=NWHIyTRUsMg
https://en.wikipedia.org/wiki/Computer_chess
https://www.vut.cz/www_base/zav_prace_soubor_verejne.php?file_id=118602
https://naos-be.zcu.cz/server/api/core/bitstreams/7d6a119b-038a-4cb4-8491-67fbfcadae4c/content
https://cs.wikipedia.org/wiki/Minimax_(algoritmus)
https://www.chessprogramming.org/Piece-Square_Tables

Obrázky:
https://i.sstatic.net/hxGdi.png